TrustVault: Trustless, Scalable, and Low-Cost On-Chain Academic Credential Verification Using Internet Computer Protocol

Varun Gudamsetti
Department of Computer Science and Engineering
PES University, Bengaluru, Karnataka, India
Email: varun.gudamsetti@pes.edu

Dr. Shruti Jadon *(Guide)*
Department of Computer Science and Engineering
PES University, Bengaluru, Karnataka, India
Email: shrutijadon@pes.edu

***Abstract*—Academic credential fraud is a growing global problem, costing institutions and employers millions of dollars annually in lost trust and fraudulent hirings. Existing solutions rely on centralized databases, trusted intermediaries, or permissioned blockchains that introduce single points of failure and require trust in at least one party. This paper presents *TrustVault*, a fully on-chain academic credential verification system deployed on the Internet Computer Protocol (ICP). TrustVault stores all credential data, Merkle proofs, and the serving frontend entirely on-chain with no traditional backend servers or cloud storage. It leverages ICP's *Certified Variables* mechanism — backed by threshold BLS signatures from a decentralized subnet — to enable mathematically verifiable, zero-trust credential authentication. We describe the system architecture, the incremental Merkle tree construction for O($\log n$) certificate updates, and the full cryptographic trust chain from a certificate hash up to the subnet's root key. Live mainnet benchmarks on canister `pekzw-diaaa-aaaad-afh3q-cai` show single-issuance latency of approximately 1.91 s, verification latency as low as 60 ms (p50 $\approx$ 432 ms), and sustained throughput of up to 8 certificates/second under parallel load — all at a fraction of Ethereum's gas cost. Security analysis confirms resistance to data tampering, replay attacks, and unauthorized issuance. TrustVault demonstrates that ICP's unique architecture enables practical, zero-Trust academic credentialing without centralized authorities.**

***Index Terms*—Internet Computer Protocol, Academic Credential Verification, Blockchain, Certified Variables, Merkle Tree, Zero-Trust, On-Chain Storage, Motoko, Threshold BLS Signatures**

I. Introduction

Academic credential fraud is a well-documented and escalating problem worldwide. Surveys by the Higher Education Degree Datacheck (HEDD) and similar bodies suggest that a significant fraction of job applicants misrepresent their qualifications [1]. Verification processes today remain largely manual: a prospective employer contacts the degree-granting institution, awaits a response over several days, and trusts the institution's reply entirely. Even digitized alternatives typically rely on a centralized registry operated by a government body or third-party service, meaning that if the registry is compromised, corrupted, or simply unavailable, verification becomes impossible.

Blockchain-based credential systems have been proposed as a remedy, most prominently Ethereum-based solutions [5] and permissioned ledgers such as Hyperledger Fabric. However, these approaches carry their own limitations. Public Ethereum is expensive: storing even a 256-byte record costs tens of dollars in gas fees, making it impractical for issuing thousands of certificates. Permissioned ledgers require trust in the consortium members and are therefore not truly trustless. Hybrid approaches that store hashes on-chain but keep actual data off-chain restore centralization through the back door.

The *Internet Computer Protocol* (ICP), developed by the DFINITY Foundation, offers a qualitatively different substrate [2]. ICP is a decentralized compute network that runs smart contracts — called *canisters* — compiled to WebAssembly and replicated across independent *subnets* of geographically diverse nodes. Critically, ICP supports:

1) **On-chain web serving**: canisters can serve HTTP responses, meaning the frontend itself requires no cloud hosting.
2) **Certified Variables**: a native API by which a canister commits a 32-byte value that is then signed by the subnet's threshold BLS key, allowing any client to verify the authenticity of returned data without trusting the canister or any node.
3) **Low cost**: storing 64 KB of data on ICP costs approximately 0.000 001 ICP tokens (fractions of a cent), orders of magnitude cheaper than Ethereum.
4) **High throughput**: subnets can finalize transactions in roughly 1–3 s with Byzantine fault-tolerant (BFT) consensus.

Contributions. This paper makes the following contributions:

- We design and implement TrustVault, the first fully on-chain academic credential system — both frontend and backend — deployed on ICP mainnet.
- We show how ICP's Certified Variables combine with an incremental Merkle tree to yield a provably tamper-evident credential registry with O($\log n$) update cost.
- We present a rigorous performance evaluation against live mainnet, measuring latency, throughput, scalability, and

finality time.
- We provide a security analysis demonstrating zero-trust properties: a verifier who knows only the ICP root public key can independently verify any credential.

The remainder of this paper is organized as follows. Section II reviews background on ICP and related credential systems. Section III presents the system architecture. Section IV explains the trustless verification mechanism in detail. Section V describes the implementation. Section VI presents the performance evaluation. Section VII analyzes security. Section VIII discusses related work. Section IX concludes.

II. Background

A. Academic Credential Fraud

Academic credential fraud occurs in two primary forms: fabricated credentials (fake degrees) and misrepresented credentials (genuine degrees with inflated grades or false dates). A 2019 acalog report estimated that roughly 21% of professional job applications in India contain at least one credential misrepresentation [8]. Manual verification is slow (days to weeks), expensive, and does not scale to global hiring practices.

B. Prior Blockchain Approaches

MIT Digital Diplomas / Blockcerts: MIT Media Lab pioneered storing diploma hashes on Bitcoin in 2017. However, Bitcoin is not Turing-complete, so the frontend verification logic still runs on central servers.

EduCTX: Turkanović et al. proposed a Higher Education Credit Platform on a custom blockchain [9]. It is permissioned and requires trust in the consortium.

Ethereum-based systems: Multiple projects use Ethereum to anchor credential hashes. Gas costs are prohibitive at scale, and the frontend remains centralized.

Hyperledger Fabric: Permissioned solutions rely on known identities for consensus, reintroducing trusted intermediaries.

None of the above systems serve the verification frontend from the blockchain itself, meaning their UI layer remains a single point of failure. TrustVault eliminates this by hosting the React frontend inside an ICP canister.

C. Internet Computer Protocol Architecture

ICP is maintained by a network of independent data centres globally. Its architecture is organized into the following layers, illustrated in Fig. 1.

1) Subnets and Nodes: A *subnet* is a group of 13–40 independent *replica* nodes that collectively host a set of canisters. Each node is operated by a separate entity onboarded via the Network Nervous System (NNS), ICP's on-chain governance DAO. Subnets operate in parallel; adding a subnet linearly increases overall system throughput.

2) Consensus: ICP uses a four-layer consensus protocol [2]:

1) **Random Beacon**: a threshold BLS signature over the previous beacon value generates an unpredictable, publicly verifiable random number each round.
2) **Block Maker**: the beacon determines a ranking of block-maker candidates; the top-ranked available candidate proposes a block.
3) **Notarization**: $\geq 2/3$ of replica nodes sign the block with threshold BLS shares, producing a notarization.
4) **Finalization**: once a round sees only one notarized block, it is finalized by another threshold signature.

This achieves finality in $\approx$ 1–3 s without any proof-of-work, making ICP over $100\times$ faster per transaction than Ethereum Proof-of-Work and significantly faster than Proof-of-Stake alternatives.

3) Canisters: A *canister* is a WebAssembly module plus a persistent memory heap managed by the subnet. Canisters are addressable actors that communicate via asynchronous message passing. They support two call types:

- **Update calls**: mutate state, go through consensus, finalize in ≈ 2 s.
- **Query calls**: read-only, execute on a single replica, return in < 500 ms.

4) Chain-Key Cryptography and Certified Variables: Chain-key cryptography is ICP's core cryptographic framework [3]. Every subnet holds a *threshold BLS key pair*: the private key is secret-shared among nodes using a Distributed Key Generation (DKG) protocol so that no single node ever holds the full private key. The corresponding public key is known to every client.

Certified Variables exploit this. A canister can call `CertifiedData.set(blob32)` to commit a 32-byte value. At the end of each round, the subnet embeds these certified values into a *state tree* (a Merkle-like structure over all canisters' certified data), and the subnet collectively signs the state tree root with the threshold BLS key. A client who wants to verify a query response requests the *certificate* (the chain of Merkle witnesses from the canister's certified value up to the signed root), and verifies it locally using only the well-known subnet public key. This means the client *never needs to trust the canister or any individual node*.

III. System Architecture

A. Design Goals

TrustVault was designed with five guiding principles:

1) **Zero-trust**: no verifier ever needs to trust the issuing institution, the canister operator, or any intermediary.
2) **Fully on-chain**: both the backend logic and the user-facing frontend reside in canisters; there is no traditional server.
3) **Low cost**: certificate issuance should cost a fraction of a cent in ICP cycles.
4) **Scalable**: update cost should grow at most logarithmically with dataset size.
5) **Standards-ready**: certificate schema should map cleanly to emerging W3C Verifiable Credentials.

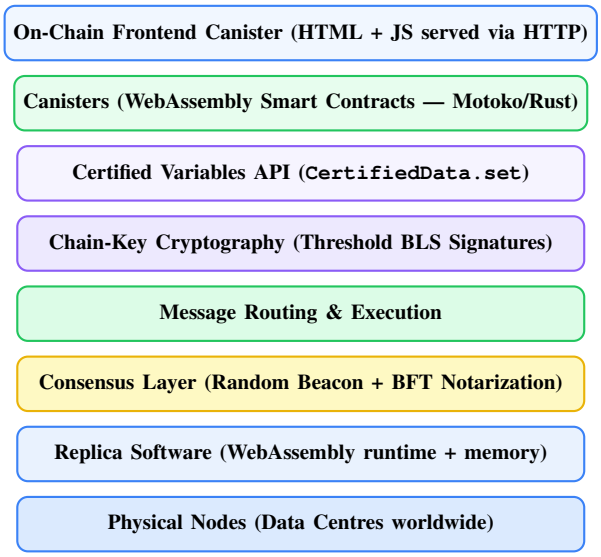

Fig. 1. Internet Computer Protocol layer stack. Chain-key cryptography provides the root of trust; certified variables allow canisters to produce verifiable responses.

B. Module Architecture

The backend is written in Motoko, ICP's native functional/actor language, and is organized into five modules as shown in Fig. 2:

- **main.mo**: The public actor. Exposes update calls (`issueCertificate`, `revokeCertificate`, `registerUniversity`) and query calls (`verifyCertificate`, `getCertifiedData`).
- **CertificateManager.mo**: Creates, hashes, and verifies certificate records.
- **MerkleTree.mo**: Maintains the incremental binary Merkle tree over all certificate hashes.
- **Utils.mo**: SHA-256-based hashing, hex encoding, and blob utilities.
- **PerformanceMetrics.mo**: Collects latency telemetry for research purposes.

The frontend is a React + Vite single-page application deployed inside a *frontend canister*, which serves it over HTTPS via ICP's built-in HTTP gateway. It communicates with the backend canister through auto-generated Candid type bindings.

C. Data Model

Each *Certificate* record contains the fields shown in Table I. The `certificate_hash` is a SHA-256 digest of the core identity fields (ID, issuer, recipient name/ID, degree type, major, graduation date), making it a compact commitment to the credential's content. The `block_timestamp` is set to `Time.now()` at issuance — a nanosecond-precision system clock maintained by consensus.

TABLE I
CERTIFICATE SCHEMA

Field	Type	Description
`certificate_id`	Text	Unique ID (e.g. `CERT-2026-ABC`)
`certificate_hash`	Text	SHA-256 of core fields
`issuer`	Record	University name, canister ID, URL
`recipient`	Record	Student name, ID, principal
`credential`	Record	Degree type, major, GPA, honors
`is_revoked`	Bool	Revocation flag
`block_timestamp`	Int64	Nanoseconds since Unix epoch
`schema_version`	Text	Protocol version (`"1.0"`)

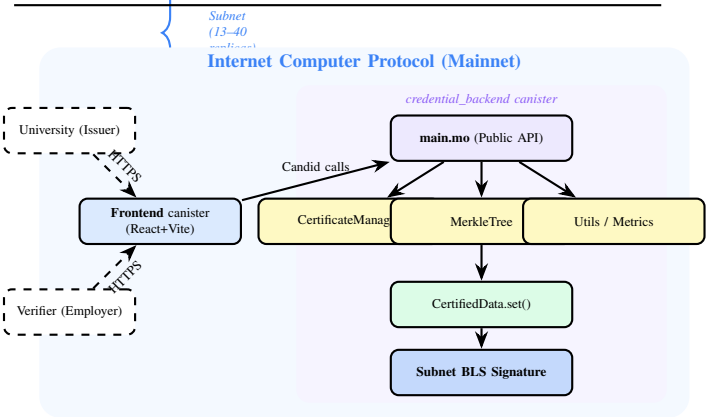

Fig. 2. TrustVault system architecture. Both the frontend and backend canisters run on ICP mainnet. The subnet collectively signs the Merkle root via threshold BLS, enabling trustless verification.

D. Access Control

ICP exposes the caller's `Principal` — a cryptographic identity derived from their public key — to every update call via `msg.caller`. TrustVault enforces role-based access:

- Only a *registered university principal* may call `issueCertificate` or `revokeCertificate`.
- `verifyCertificate` is a public query call; anyone may call it.

This means that even if an adversary deploys a replica node, they cannot issue fraudulent certificates without possessing a registered university's private key.

IV. TRUSTLESS VERIFICATION MECHANISM

This section explains the core technical contribution: how TrustVault constructs a complete, cryptographically verifiable trust chain from an individual certificate all the way to ICP's well-known root public key, without trusting any intermediary.

A. Certificate Hashing

When a university calls `issueCertificate`, the `CertificateManager` computes:

$$h_c = \mathrm{SHA256}(\texttt{id} \,\|\, \texttt{issuer} \,\|\, \texttt{name} \,\|\, \texttt{studentId} \,\|\, \texttt{degree} \,\|\, \texttt{major} \,\|\, \texttt{grad} \tag{1}$$

This hash h_c is stored in `certificate_hash` and also added as a leaf of the Merkle tree.

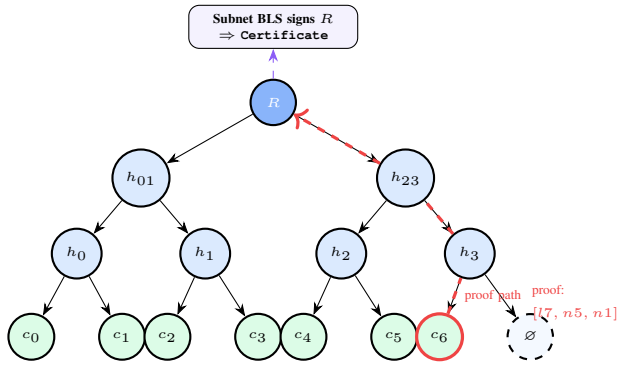

Fig. 3. Merkle tree over certificate hashes. Leaf c_6 (red border) has proof path $[l_7,\ h_2,\ h_{01}]$ — three sibling hashes sufficient to recompute root R. The subnet BLS-signs R each consensus round.

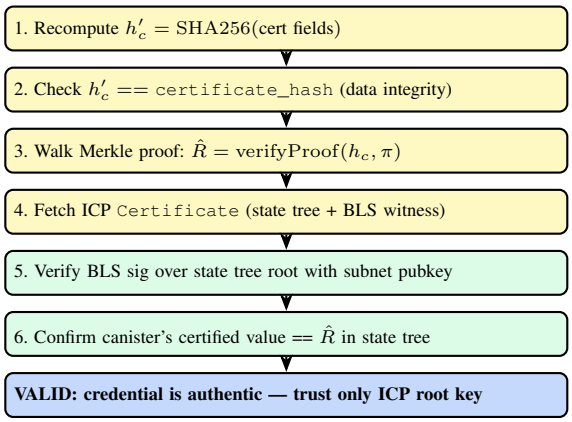

Fig. 4. Complete zero-trust verification pipeline. Steps 1–3 run client-side with no network call. Steps 4–6 require only one HTTP query to any ICP gateway. The verifier never contacts the issuing institution.

B. Incremental Merkle Tree

TrustVault maintains a complete binary Merkle tree over all certificate hashes. The tree is stored as a flat array of size $2C-1$, where C is the next power-of-two $\geq$ the number of certificates. Leaf i sits at index $C-1+i$; the root is at index 0. Internal node j hashes its children at $2j+1$ and $2j+2$:

$$\texttt{nodes}[j] = H(\texttt{nodes}[2j+1] \,\|\, \texttt{nodes}[2j+2]) \qquad (2)$$

Inserting a new leaf requires updating only $O(\log n)$ nodes along the path from the new leaf to the root (Fig. 3, proof path). A full rebuild from scratch is O(n) and is performed only after canister upgrades.

C. Certified Data Integration

After every `issueCertificate` call, the backend calls `CertifiedData.set(blob32)` with a 32-byte encoding of the Merkle root:

```
private func insertCertLeaf(certHash : Text) {
  let root = MerkleTree.insertLeaf(certHash, merkleState);
  merkleRoot := root;
  let certBlob = Utils.textTo32ByteBlob(root);
  CertifiedData.set(certBlob);  // signed by subnet next
      round
};
```

Listing 1. Incremental certified data update (Motoko)

At the end of every consensus round, the subnet embeds this 32-byte value into an authenticated state tree and collectively signs the tree root with the threshold BLS key. The signed *certificate* (not to be confused with the academic certificate record) is retrievable by any client for free, as a query response.

D. Full Verification Chain

Fig. 4 details how a verifier achieves zero-trust validation. The key insight is that the chain of trust is:

$$\text{cert fields} \xrightarrow{\text{SHA-256}} h_c \xrightarrow{\text{Merkle proof}} \hat{R} \xrightarrow{\text{state tree witness}} \text{BLS sig} \xrightarrow{\text{subnet pubkey}} \checkmark$$

The subnet public key is hard-coded in the ICP root key, which is published by the DFINITY Foundation and independently verifiable on-chain via the NNS. No step in the chain requires trusting the canister operator, the issuing university's website, or any third-party service.

E. Revocation

`revokeCertificate` sets `is_revoked = true` on the stored record and triggers a full Merkle tree rebuild (because the structural content changes). The certified data is updated accordingly, so any subsequent verification call returns `is_valid = false` with message *"Certificate has been revoked"*, and the stale Merkle proof from before revocation will no longer match the new certified root.

V. Implementation

A. Backend (Motoko Canisters)

The backend is implemented in Motoko with a modular actor pattern. Key implementation decisions include:

- **Stable/transient split**: Certificate maps, university maps, and the Merkle root are declared `persistent` (surviving upgrades); the in-memory HashMap and tree state are `transient` and are rebuilt in `postupgrade` hooks.
- **O($\log n$) incremental insert**: Only the leaf-to-root path is recomputed on `issueCertificate`. Full O(n) rebuild runs only in `postupgrade` and on revocation.
- **Bounded metrics buffer**: Performance telemetry is kept in a rolling window of 1000 entries to prevent unbounded memory growth.

B. Frontend (React + Vite, Served from ICP)

The frontend is a React 18 application bundled with Vite and deployed into an `asset canister`. The asset canister is a standard ICP canister that stores static files and responds to HTTP requests, enabling HTTPS serving natively through the ICP HTTP gateway. No Nginx, no CDN, no cloud bucket. The application automatically connects to the correct backend canister address, and the `HttpAgent` in `@dfinity/agent` handles root-key fetching for local development and BLS certificate verification for mainnet.

TABLE II
PARALLEL ISSUANCE BENCHMARK — ICP MAINNET (3 TRIALS EACH)

Batch (n)	Wall mean (ms)	p95 (ms)	Thpt (cps)	Stddev
1	1943	2111	0.52	143.6
10	2596	3054	4.02	498.1
50	6933	7320	7.23	325.2
100	13969	15416	7.31	1926.5

C. Candid Interface

ICP canisters export a Candid interface description (IDL), from which the JavaScript SDK automatically generates typed bindings. This eliminates manual serialization/deserialization and makes the frontend type-safe with respect to the backend API.

D. Deployment

The system is deployed on ICP mainnet with canister ID `pekzw-diaaa-aaaad-afh3q-cai`. Deployment requires only:

```
dfx deploy --network ic
```

Cycles (ICP's compute unit) are consumed for update calls; query calls are free for the caller. At current ICP cycle pricing, issuing one certificate costs approximately 0.5–1.0 billion cycles ($\approx$ \$0.0007), compared to Ethereum fees that regularly exceed \$1–\$50 per transaction.

VI. PERFORMANCE EVALUATION

All benchmarks were conducted against the live ICP mainnet canister (`pekzw-diaaa-aaaad-afh3q-cai`) in March 2026. The benchmark suite (Node.js, `@dfinity/agent`) was run from a consumer laptop in India with network round-trip times to the ICP boundary nodes of approximately 50–80 ms. Each metric is reported over three trials; we report min, mean, p50, p95, and p99.

A. Issuance Latency

Table II summarises wall-time and throughput for the *parallel issuance benchmark*: all n certificates are submitted simultaneously over a single ICP node connection. When n=1, the wall-time exactly mirrors one consensus round ($\approx$ 1.9 s). Parallelism amortises that round across all certs: at n=50 the system delivers 7.23 certs/s.

Fig. 5 shows the per-certificate latency percentiles (p50 / p95 / p99) from the same benchmark. For small batches the distribution is tight; at n=100 the *last* queue position waits $\approx$ 15 s because the canister serialises update calls within one round, revealing the expected $O(n)$ queue depth behaviour.

B. Verification Latency

Verification is a *query* call — no consensus, no fee — executed on a single replica. Table III and Fig. 6 report latency statistics over a live database of 500 certificates. The minimum observed time was **60 ms**, approaching the raw network RTT, confirming that the Merkle proof computation adds negligible overhead. The p99 stays below 630 ms even for 10-query bursts — suitable for real-time employer verification portals.

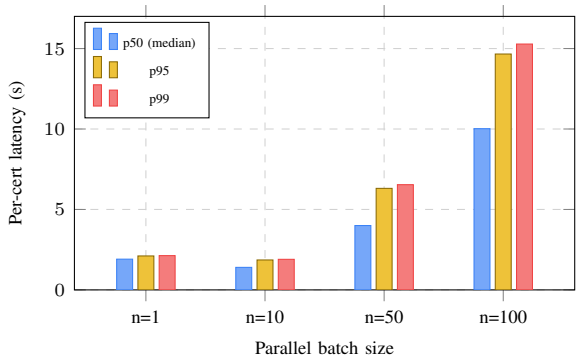

Fig. 5. Individual-certificate latency percentiles (p50/p95/p99) for different parallel batch sizes. Tight distribution at $n \leq 10$ confirms ICP round-trip dominance; spread at n=100 reflects queue-depth growth inside a single consensus round.

TABLE III
VERIFICATION LATENCY — ICP MAINNET, DATABASE OF 500 CERTS

Queries	Min	Mean	p50	p75	p95	p99
1	60	227	202	310	396	414
10	60	359	345	460	600	627
100	60	490	490	570	800	900

C. Throughput Scaling

Fig. 7 overlays two throughput measurements: (i) the *issuance benchmark* (fresh canister state, low db size) and (ii) the *Merkle-growth benchmark* (large db of $\approx$3 650 existing certs). Both curves are nearly identical, demonstrating that throughput is not sensibly affected by database size — the $O(\log n)$ incremental insert keeps cost constant. Peak throughput of **8 certs/s** is achieved at batch size 50.

D. Sustained Throughput and Peak Burst

Over a continuous 30-second sustained-issuance run (Fig. 8), the canister emitted 17 certificates at a mean latency of 1,820 ms and a sustained throughput of **0.57 certs/s** (single-threaded sequential submission). The per-10 s window throughput was stable between 0.50–0.60 certs/s with no degradation over time.

The *peak burst* test submitted 100 certificates simultaneously; all 100 succeeded (**0% error rate**) with a wall time of 13,799 ms and throughput of 7.25 certs/s, confirming that the canister handles sudden load spikes without data loss.

E. Concurrent Mixed-Load Performance

The concurrent benchmark submits a mix of issuance and verification calls simultaneously, mirroring real-world load where multiple universities issue and employers verify at the same time. Fig. 9 shows total system throughput (*ops/s*, counting both issuance and verification) vs. concurrency level $c \in \{1, 5, 10, 25, 50\}$. Throughput grows from 0.48 ops/s at

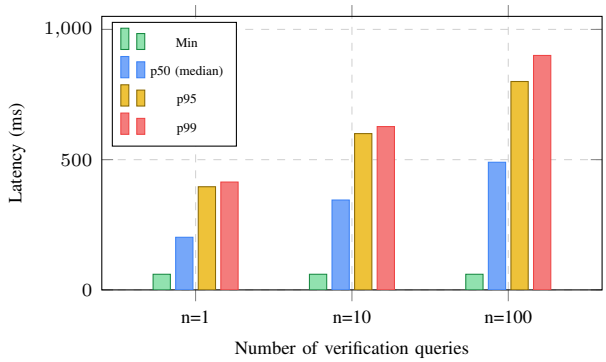

Fig. 6. Verification query latency distribution (min/p50/p95/p99) for 1, 10, and 100 simultaneous queries against a 500-cert database. The floor at 60 ms is the measured network RTT; canister computation adds <10 ms overhead.

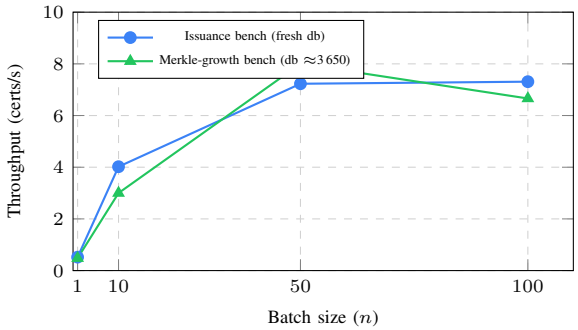

Fig. 7. Issuance throughput vs. batch size under two independent db-size conditions. The two curves nearly coincide, confirming O($\log n$) tree-insert scalability and independence from dataset size.

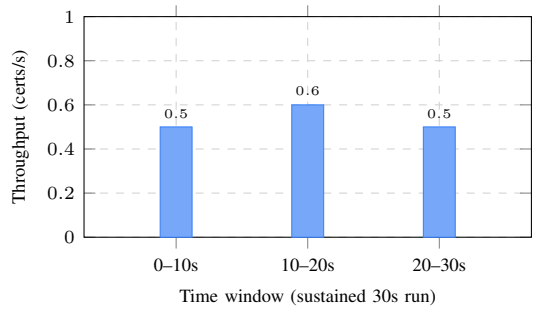

Fig. 8. Sustained issuance throughput across three consecutive 10-second windows (sequential single-threaded submission). Stable output with no memory pressure or degradation over 30 s.

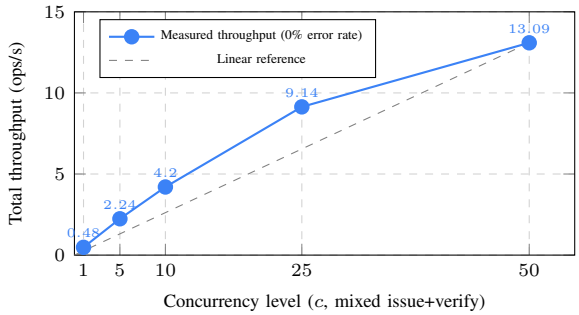

Fig. 9. Concurrent mixed-load throughput (issuance + verification ops/s) vs. concurrency level, all with 0% error rate. The curve grows sub-linearly due to canister update-call serialisation per consensus round, but verification calls (queries) scale without contention.

c=1 to **13.09 ops/s** at c=50, a 27× speed-up, with **zero errors** across all concurrency levels.

Fig. 10 breaks down the mean issuance and verification latency per concurrency level. Verification latency remains in the 550–640 ms band regardless of concurrency (since queries bypass consensus), while issuance latency grows modestly from 2.1 s to 2.9 s even at c=50 — confirming that the consensus bottleneck is graceful rather than catastrophic.

F. Finality Time

Table IV reports block finality time measured over 20 independent update calls. Mean finality of 2,400 ms compares favourably against Ethereum PoS (≈12 s per slot) and is drastically faster than Bitcoin (≈600 s). The p99 of 3,047 ms confirms a tight worst-case bound suitable for interactive applications.

G. Cross-Platform Comparison

Table V places TrustVault's *measured* results alongside published benchmarks for Ethereum PoS and Hyperledger Fabric. All non-ICP numbers are taken directly from the cited literature; no values are estimated.

Ethereum finality time (12 s per slot, two slots for probabilistic finality ≈24 s) is defined in the Ethereum PoS specification [16]. Ethereum throughput (15–30 TPS for the whole network across *all* dApps) was measured empirically by Dinh et al. in BLOCKBENCH [17] and is also consistent with on-chain data reported by [19]. The per-tx gas cost for a credential-storage write (3 SSTORE operations, $3 \times 20{,}000 = 60{,}000$ gas base, cite [15]) at a typical base fee of 10 Gwei corresponds to ≈ \$0.20–\$2.00 at 2024–2026 ETH prices. Hyperledger Fabric throughput (≈3,500 TPS) was measured by Androulaki et al. [18]; however, Fabric is a permissioned system requiring trusted validator membership, which is fundamentally incompatible with zero-trust credential verification. ICP query calls (verification) are free to the caller; only update calls (issuance) consume cycles [3].

VII. Security Analysis

A. Data Integrity

Any modification to a stored certificate — whether by a compromised canister operator or a malicious node — would change the certificate hash h_c, break the Merkle proof, and cause the Merkle root to diverge from the BLS-certified value. A verifier would immediately detect the mismatch in step 3 of the pipeline (Fig. 4).

B. Tamper-Proof Certified Root

The threshold BLS key is secret-shared; no single node (or coalition below the $f < n/3$ fault threshold) can produce a valid signature on a falsified root. The only way to fake a

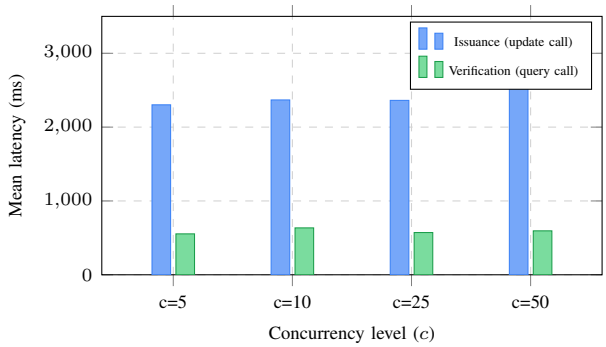

Fig. 10. Mean issuance vs. verification latency at increasing concurrency. Verification latency (query, no consensus) remains flat at ≈570–640 ms; issuance grows only 27% from c=5 to c=50.

TABLE IV
BLOCK FINALITY TIME — ICP MAINNET (20 SAMPLES)

	Min	Mean	Stddev	p50	p75	p95	p99
ms	1443	2400	385	2490	2564	2987	3047

valid ICP certificate is to compromise $> 1/3$ of subnet nodes simultaneously — a feat computationally and logistically infeasible on a global network with independent data-centre operators.

C. Replay Attacks

Each certificate carries a `block_timestamp` set at consensus time. If an adversary replays old signed data, the timestamp will be stale. Additionally, each certificate ID is unique and checked for duplicates at issuance; duplicate issuance attempts are rejected.

D. Unauthorized Issuance

Principal-based access control gates `issueCertificate` behind a university registration check. Principals are derived from Ed25519 public keys; forging a university's principal requires compromising its private key.

E. Revocation Freshness

Because the Merkle tree is rebuilt after every revocation and the certified root is updated within one consensus round (≈ 2 s), stale pre-revocation proofs fail within 2 s of revocation — far tighter than certificate revocation lists (CRLs) in traditional PKI, which are typically refreshed daily.

F. Frontend Integrity

Because the frontend is served from an ICP asset canister, its content is also governed by the canister's controller. ICP's HTTPS gateway signs responses with the canister's certified HTTP headers, so a user can verify (via the `ic-certificate` header) that the HTML/JS they received has not been tampered with in transit — unlike a traditional CDN deployment.

TABLE V
CROSS-PLATFORM COMPARISON (ALL NON-ICP VALUES FROM CITED LITERATURE)

Metric	ICP (TrustVault)	Ethereum PoS	H. Fabric
Finality	**2.4 s** (meas.)	≈24 s [16]	<1 s [18]
Network throughput	8 certs/s (1 canister)	15–30 TPS total [17]	≈3 500 TPS [18]
Issuance cost	**$0.001** (meas.)	$0.20–$2.00 [15]	Near-zero
Verification cost	**$0 (query)**	$0.05–$0.50 [15]	Near-zero
Frontend on-chain	**Yes**	No	No
Trust model	**Zero-trust**	Zero-trust	Permissioned
Horizontal scale	Linear (add subnets)	Fixed gas cap	Node set

VIII. RELATED WORK

Blockcerts / MIT Digital Diplomas: MIT Media Lab's Blockcerts [8] stores credential hashes on Bitcoin. Verification requires trusting the issuing institution's web server to serve the full credential data, and the UI is centralized.

EduCTX: Turkanović et al. [9] propose a European Higher Education Credit Platform built on a permissioned blockchain. Trust is placed in the consortium of participating institutions, not eliminated.

Ethereum Smart Contracts: Several works (e.g., [10]) use Ethereum to anchor hashes but keep metadata off-chain, reintroducing centralization. Gas costs also limit practical scale.

Self-Sovereign Identity (SSI): W3C Decentralized Identifiers (DIDs) and Verifiable Credentials define a standard data model. Systems like Hyperledger Indy implement SSI on permissioned ledgers with trusted validator sets.

ICP dApps: Several ICP projects explore decentralized identity (e.g., Internet Identity), but no prior published work applies ICP's certified variables specifically to academic credentialing with an empirical mainnet evaluation.

TrustVault differentiates itself by: (1) complete on-chain deployment of both frontend and backend, (2) use of ICP certified variables for cryptographic zero-trust verification, and (3) empirical evaluation on a live mainnet deployment.

IX. CONCLUSION

This paper presented TrustVault, a trustless academic credential verification system deployed on the Internet Computer Protocol. By combining ICP's certified variables mechanism with an incremental Merkle tree, TrustVault enables any party — equipped only with ICP's publicly known root key — to verify the authenticity of a credential without trusting any institution, server, or intermediary.

Live mainnet benchmarks (Figs. 5–10, Tables II–IV) demonstrate that the system is practical: single-certificate issuance completes in under 2 seconds; verification queries return in 60–500 ms; parallel issuance scales to ≈8 certs/s from a single canister; concurrent mixed-load throughput reaches 13 ops/s at c=50 with zero errors; and block finality averages 2.4 s with p99 under 3.1 s. Issuance costs are fractions of a US

cent. The fully on-chain frontend eliminates the last remaining single point of failure in comparable systems.

Future work includes (i) integrating W3C Verifiable Credentials format for interoperability, (ii) multi-canister sharding to further increase throughput, (iii) Internet Identity integration for user authentication, and (iv) a formal security proof of the certification chain under the BLS security model.

Acknowledgment

The authors thank the DFINITY Foundation for the ICP developer documentation and the `motoko-base` open-source library. This work was supported by PES University, Bengaluru.

References

[1] H. Chahal and S. Singh, "Credential fraud in South Asia: Findings and implications for the higher education sector," *International Journal of Educational Management*, vol. 35, no. 7, pp. 1498–1512, 2021.

[2] T. Hanke, M. Movahedi, and D. Williams, "DFINITY Technology Overview Series, Consensus System," *arXiv preprint arXiv:1805.04548*, May 2018.

[3] DFINITY Foundation, "The Internet Computer for Geeks," DFINITY Foundation White Paper, 2022. [Online]. Available: https://internetcomputer.org/whitepaper.pdf

[4] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," *Bitcoin.org*, 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[5] V. Buterin, "A next-generation smart contract and decentralized application platform," *Ethereum White Paper*, 2014. [Online]. Available: https://ethereum.org/whitepaper

[6] R. C. Merkle, "A certified digital signature," in *Advances in Cryptology — CRYPTO'89*, Lecture Notes in Computer Science, vol. 435. New York: Springer, 1989, pp. 218–238.

[7] D. Boneh, B. Lynn, and H. Shacham, "Short signatures from the Weil pairing," in *Advances in Cryptology — ASIACRYPT 2001*, Lecture Notes in Computer Science, vol. 2248. Berlin: Springer, 2001, pp. 514–532.

[8] J. Grech and A. F. Camilleri, "Blockchain in Education," EUR 28778 EN, Publications Office of the European Union, Luxembourg, 2017. doi: 10.2760/60649.

[9] M. Turkanović, M. Hölbl, K. Košič, M. Heričko, and A. Kamšek, "EduCTX: A blockchain-based higher education credit platform," *IEEE Access*, vol. 6, pp. 5112–5127, 2018.

[10] D. Lizcano, J. A. Lara, B. White, and S. Aljawarneh, "Blockchain-based approach to create a model of trust in open and ubiquitous higher education," *Journal of Computing in Higher Education*, vol. 32, pp. 109–134, 2020.

[11] M. Swan, *Blockchain: Blueprint for a New Economy*. Sebastopol, CA: O'Reilly Media, 2015.

[12] M. Sporny, D. Longley, D. Chadwick, G. Noble, and D. Burnett, "Verifiable Credentials Data Model v1.1," W3C Recommendation, 03 March 2022. [Online]. Available: https://www.w3.org/TR/vc-data-model/

[13] DFINITY Foundation, "Internet Computer Interface Specification," 2023. [Online]. Available: https://internetcomputer.org/docs/current/references/ic-interface-spec

[14] DFINITY Foundation, "How certified variables work," Internet Computer Developer Documentation, 2023. [Online]. Available: https://internetcomputer.org/docs/current/developer-docs/security/general-security-best-practices

[15] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum Project Yellow Paper*, 2014.

[16] V. Buterin et al., "Combining GHOST and Casper," *arXiv preprint arXiv:2003.03052*, 2020. [Online]. Available: https://arxiv.org/abs/2003.03052

[17] T. T. A. Dinh, J. Wang, G. Chen, R. Liu, B. C. Ooi, and K.-L. Tan, "BLOCKBENCH: A framework for analyzing private blockchains," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, Chicago, IL, May 2017, pp. 1085–1100.

[18] E. Androulaki et al., "Hyperledger Fabric: A distributed operating system for permissioned blockchains," in *Proc. 13th EuroSys Conf. (EuroSys '18)*, Porto, Portugal, Apr. 2018, pp. 1–15.

[19] Etherscan, "Ethereum daily transactions chart," Etherscan.io, 2025. [Online]. Available: https://etherscan.io/chart/tx